# Ders 22 Çalışma Özeti

## Ders 22 Çalışma Özeti

### Genel Konular

- Quiz sorularının çözümü: minimum yayılan ağaç (minimum spanning tree, MST), en küçük elemanı silmenin karmaşıklığı, böl ve yönet (divide and conquer) uygunluğu, bağlı bileşenler (connected components)
- Böl ve yönet yaklaşımının şartları: parçaların birbirinden bağımsız çözülebilmesi ve bölmenin kazanç sağlaması
- Hızlı sıralama (quicksort): pivot seçimi, değere göre bölme (partitioning)
- `partition` fonksiyonu: iki indisle bölme, son yer değiştirmeyi geri alma (undo last swap)
- Quicksort karmaşıklığı: en iyi durum, ortalama durum, en kötü durum
- En kötü durumdan kaçınma: diziyi karıştırma (shuffle), rastgele pivot, üç elemanın medyanı
- Insertion sort, mergesort ve quicksort çalışma süresi karşılaştırması
- Küçük parçalarda insertion sort'a geçme ve mergesort için bir iyileştirme
- Veri sıkıştırma ve run-length encoding (RLE)

### Hocanın Özellikle Vurguladığı Kısımlar

- Kruskal algoritmasında kenarlar artan ağırlık sırasıyla işlenir, çünkü doğru sonucu (minimum ağacı) bu sıra garanti eder. "Minimum spanning tree" denince akla önce en küçük ağırlıklar gelmelidir.
- Dizide silme, gözün içine 0 yazmak değildir. Elemanlar kaydırılır ve dizi artık $n - 1$ elemanlı kabul edilir.
- Her özyinelemeli (recursive) algoritma böl ve yönet değildir. Faktöriyel ve Fibonacci özyinelemeli yazılabilir ama böl ve yönet olamaz. Hoca gerekçe olarak her adımın bir önceki sonuca ihtiyaç duyduğunu söyler (faktöriyel için bkz. ders sonrası notlar).
- Böl ve yönet için iki şart vardır:
  - Parçalar birbirinden bağımsız çözülebilmelidir.
  - Bölmek gerçek bir kazanç sağlamalıdır ("attığın taş ürküttüğün kurbağaya değsin"). Sıralamada $n^2$'den $n \log n$'e inmek böyle bir kazançtır.
- Mergesort elemanları dizideki **yerlerine** göre böler. Quicksort ise elemanları **değerlerine** göre (pivottan küçük/büyük) böler.
- Quicksort'ta pivot, bakılan bölgenin ilk elemanı (`a[l]`) olarak seçilir. Pivot önce hiç kıpırdatılmaz. Diğer elemanlar ondan küçükler ve büyükler diye ayrıldıktan sonra pivot yerine konur.
- Son yer değiştirmeyi döngünün içinde `if (i < j)` ile engellemek yerine döngüden sonra bir kez geri almak daha verimlidir. Bir milyon elemanlı dizide `if` bir milyon kez çalışır; geri alma ise yalnız bir kez yapılır.
- Quicksort'un en kötü durumu sıralı dizidir. Pivot her seferinde bir uca düşer ve karmaşıklık $n^2$ olur.
- Quicksort ek dizi kullanmadığı için büyük dizilerde en çok tercih edilen sıralama algoritmasıdır.
- Küçük parçalarda özyinelemenin maliyeti yüzünden insertion sort daha avantajlıdır. Bu yüzden pratik quicksort uygulamaları küçük parçaları insertion sort ile sıralar.

## Kısa Tekrar Notları

- $n$ elemanlı yapıda en küçük elemanı silmenin en kötü durum karmaşıklığı:

| Yapı | Karmaşıklık | Neden |
|---|---|---|
| Küçükten büyüğe sıralı dizi | $O(n)$ | Baştaki silinir, geri kalan herkes bir sola kayar |
| Sırasız dizi | $O(n)$ | Önce bulunur ($n$), sonra kaydırılır ($n$); $2n$ yine $O(n)$ |
| Küçükten büyüğe sıralı tek yönlü bağlı liste | $O(1)$ | En küçük başta; `head` bir sonrakini gösterir, düğüm `free` edilir |
| Sırasız tek yönlü bağlı liste | $O(n)$ | Sona kadar bakmak gerekir |
| İkili min-heap | $O(\log n)$ | Son düğüm köke alınır, aşağı doğru heapify yapılır |

- Böl ve yönetle çözülmeye uygun mu?
  - Yığına eleman ekleme: hayır
  - Kuyruğa eleman ekleme: hayır
  - $n$'e kadar Fibonacci sayılarını bulma: hayır
  - Diziyi küçükten büyüğe sıralama: evet (mergesort, quicksort)
  - Faktöriyel hesabı: hayır
- Quicksort iskeleti:
  - `l < r` ise `s = partition(A[l..r])`, sonra `QuickSort(A[l..s-1])` ve `QuickSort(A[s+1..r])`.
  - İlk çağrıda `l = 0`, `r = n-1`.
  - `l == r` tek eleman demektir; zaten sıralıdır. `l > r` ise parça boştur (örneğin pivot bölgenin ilk gözüne yerleşince `QuickSort(A[l..s-1])`).
- Karmaşıklıklar:
  - En iyi durum: $C_{best}(n) = 2C_{best}(n/2) + n$, $C_{best}(1) = 0$, sonuç $n \log_2 n$
  - Ortalama durum: yaklaşık $1{,}39\, n \log_2 n$
  - En kötü durum (sıralı dizi): $\Theta(n^2)$
- En kötü durumdan kaçınmanın üç klasik yolu:
  - Başta diziyi karıştırmak (shuffle)
  - Her adımda pivotu rastgele seçmek
  - Rastgele üç elemanın medyanını pivot yapmak (pivot önce bölgenin başına alınır)
- Çalışma süreleri (saniyede $10^8$ karşılaştırma yapan bilgisayar, ekrandan, 02:31:00):

| Algoritma | Bin eleman | Milyon eleman | Milyar eleman |
|---|---|---|---|
| Insertion sort | anlık | 2,8 saat | 317 yıl |
| Mergesort | anlık | 1 saniye | 18 dakika |
| Quicksort | anlık | 0,6 saniye | 12 dakika |

## Detaylı Açıklamalar

### Quiz Sorularının Çözümü

**Kruskal ve MST İfadeleri** Soru: Kenar ağırlıkları pozitif ve birbirinden farklı (distinct) tam sayılar olan, kendine döngüsü (self-loop) ve paralel kenarı olmayan bir $G$ grafı veriliyor. Kruskal ile bulunan MST için hangileri doğrudur?

- "MST en küçük ağırlıklı kenarı mutlaka içerir": **Doğru.** Kruskal ilk olarak bu kenarı seçer; bu kenar döngü oluşturamaz.
- "MST en büyük ağırlıklı kenarı içermez": **Yanlış.** Bir düğüme yalnızca o kenarla ulaşılıyorsa en büyük kenar da ağaca girmek zorundadır.
- "$G$'nin her zaman tek bir MST'si vardır": **Doğru.** (Gerekçesi için "Ders sonrası notlar" bölümüne bakın.)

- "Bir $v$ düğümüne bağlı kenarların hepsi 10 artırılırsa eski MST yeni grafın da MST'sidir": **Yanlış.** Bu kenarların sıralamadaki yeri değişir; ağaç değişebilir.
- "Tüm kenarlar 10 artırılırsa eski MST yeni grafın da MST'sidir": **Doğru.** Göreli sıralama değişmez.

Kenarların azalan sırayla işlenmesi de sonunda çözüme götürebilirdi. Ancak seçilenleri sonradan bırakmak gerekir ve algoritma çok karmaşıklaşır. Bu yüzden "en uygun" cevap "doğruluğu garanti etmek"tir.

**Bağlı Bileşen Sorusu** Kenarlar: `1-2`, `3-4`, `5-6`, `7-8`, `7-9`, `2-8`, `0-5`, `1-9`. Bu kenarlar birleştirildiğinde 3 bağlı bileşen oluşur:

- $\{1, 2, 7, 8, 9\}$
- $\{3, 4\}$
- $\{0, 5, 6\}$

### Böl ve Yönet: Fibonacci ve Faktöriyel

- Fibonacci dizisi: 1, 1, 2, 3, 5, 8, 13, … 13'ü bulmak için 5'i ve 8'i bilmek gerekir.
- "İlk yedisini bir yerde, sekizinciden on altıncıya kadarını başka yerde bulalım" denemez. İkinci parça birinci parçanın sonucuna bağlıdır.
- Hoca faktöriyelin de aynı olduğunu söyler: $6!$ için $5!$ bilinmelidir (00:24:26). Bu gerekçe için ders sonrası notlara bakın.
- Mergesort'ta ise iki yarının sıralanması birbirinden bağımsızdır. Böl ve yönetin mantığı budur: işi parçalara böl, her parçayı kendi içinde çöz, sonra birleştir.

### Quicksort'un Fikri

Quicksort, "20. yüzyılın en önemli 10 algoritması" listelerinde geçen bir algoritmadır.

- Bir pivot seçilir. Dizi, pivottan küçükler solda ve büyükler sağda kalacak şekilde düzenlenir.
- Pivot artık dizi sıralı olsaydı bulunacağı yerdedir (`A[s]`). Solundakiler `A[0..s-1]` pivottan küçük, sağındakiler `A[s+1..n-1]` pivottan büyük ya da eşittir.
- Aynı işlem sol ve sağ parça için ayrı ayrı tekrarlanır. Her bölgenin pivotu yerine oturunca dizinin tamamı sıralanmış olur.
- Kazanç şuradan gelir: insertion ve selection sort her adımda dizinin tamamını işler. Quicksort'ta ise pivotun solunu sıralarken sağına hiç bakmak gerekmez.

Quicksort iskeleti (ekrandan, 01:01:20):

```
QuickSort(A[l..r])
  if (l < r)
    s <- partition(A[l..r])      // s: pivotun yeni adresi
    QuickSort(A[l..s-1])
    QuickSort(A[s+1..r])
ilk çağrı: l = 0, r = n-1
```

### Partition: İki İndisle Bölme

İlk akla gelen yöntem, elemanları pivotla karşılaştırıp kaydırmaktır. Ancak kaydırma çok iş çıkarır. Daha verimli yol şudur:

- Pivot (`a[l]`) yerinde bekler.
- `i` soldan sağa ilerler ve pivottan küçük elemanları atlar. Büyük bir eleman bulunca durur.
- `j` sağdan sola ilerler ve pivottan büyük elemanları atlar. Küçük bir eleman bulunca durur.
- `a[i]` ile `a[j]` yer değiştirir. Böylece küçük sola, büyük sağa geçer.
- `i` ile `j` karşılaşınca ya da birbirini geçince döngü biter.
- Son tur fazladan bir yer değiştirme yapmıştır; bu geri alınır.
- Son olarak pivot (`a[l]`) ile `a[j]` yer değiştirir ve `j` döndürülür.

Tahtadaki kod (ekrandan, 02:07:10):

```c
int partition(int a[], int l, int r)
{
    int pivot, i, j;
    pivot = a[l];
    i = l; j = r + 1;
    do
    {
        do
        {
            i++;
        } while ((i < j) && (a[i] < pivot));
        do
        {
            --j;
        } while (a[j] >= pivot);
        xchg(&a[i], &a[j]);
    } while (i < j);
    xchg(&a[i], &a[j]);   /* undo last swap */
    xchg(&a[l], &a[j]);
    return j;
}
```

Bu kod dersteki iki örnekte doğru sonuç verir. Uç durumlardaki iki sınır hatası ve düzeltilmiş, test edilmiş hâli "Ders sonrası notlar" bölümündedir.

Örnek 1: `3 2 4 7 1 8 5`, pivot 3.

| Adım | `i` durur | `j` durur | Yer değiştirme sonrası dizi |
|---|---|---|---|
| 1. tur | i=2 (4) | j=4 (1) | `3 2 1 7 4 8 5` |
| 2. tur | i=3 (7) | j=2 (1) | `3 2 7 1 4 8 5` (i > j, döngü biter) |
| Son turu geri al `a[l]` ile `a[j]` | | | `3 2 1 7 4 8 5` → `1 2 3 7 4 8 5`, `return 2` |

Sonuçta 3'ten küçükler (1, 2) solda, büyükler (7, 4, 8, 5) sağda kalır.

Örnek 2: `5 2 6 3 7 8`, pivot 5.

- `i` 6'da durur. `j` 8 ve 7'yi geçer, 3'te durur. 6 ile 3 yer değiştirir: `5 2 3 6 7 8`.
- Sonraki turda `i` 6'da (indis 3), `j` 3'te (indis 2) durur. `i > j` iken yine yer değiştirilir: `5 2 6 3 7 8`.
- Son yer değiştirme geri alınır: `5 2 3 6 7 8`.
- Pivot ile `a[j]` yer değiştirir: `3 2 5 6 7 8`, pivot 2. indiste.

Döngünün son turu her zaman fazladan bir yer değiştirme yapar. Bu yer değiştirme `i == j` ya da `i > j` iken yapılır ve küçük elemanı sağa atar. Geri alınmazsa pivot doğru yere konamaz.

Ders sırasında hoca örneği adım adım yürütürken indisleri yanlış yazdı ve algoritmada hata olduğunu düşündü. Aradan sonra algoritmanın doğru olduğunu, hatanın elle izlemede yapıldığını gösterdi. Bir öğrenci de, iki döngüde birden <= ve >= kullanılırsa eşit elemanların iki tarafta da tutulacağını hatırlattı. Hoca bunu düzeltti.

### Quicksort Karmaşıklığı

**En İyi Durum** Pivot her seferinde tam ortaya oturur. Quicksort yarı boyutta iki kez çağrılır. Partition ise $n$ karşılaştırma yapar.

Geri yerine koyma (backward substitution) ile çözüm (ekrandan, 02:16:55):

$$C(n) = 2C(n/2) + n$$

$$= 2\left[2C(n/4) + n/2\right] + n = 4C(n/4) + 2n$$

$$= 4\left[2C(n/8) + n/4\right] + 2n = 8C(n/8) + 3n$$

$$= 2^i C(n/2^i) + i \cdot n$$

$n = 2^i$ alınırsa $C(n/2^i) = C(1) = 0$ ve $i = \log_2 n$ olur:

$$C_{best}(n) = n \log_2 n$$

Bu, mergesort'un karmaşıklık hesabının aynısıdır.

**Ortalama ve En Kötü Durum**

- Ortalama durum: yaklaşık $1{,}39\, n \log_2 n$ (hoca 1,38 dedi). Ayrıntısı Algoritma Analizi dersinde işlenir.
- En kötü durum: dizi zaten sıralıysa, örneğin `1 2 3 4 5 6`.
  - Pivot 1 seçilir. Tüm elemanlar ondan büyüktür. Bir tarafta $n - 1$ eleman kalır.
  - Sonra pivot 2 seçilir. Yine bir tarafta $n - 2$ eleman kalır.
  - Parçalar hiç dengelenmez. Karmaşıklık $\Theta(n^2)$ olur.

**En Kötü Durumdan Kaçınma**

- **Shuffle:** Başta dizi bir kez karıştırılır. Sıralı dizinin düzeni bozulur.
- **Rastgele pivot:** Her adımda bakılan bölgeden rastgele bir eleman pivot seçilir.
- **Üç elemanın medyanı (median of sample):** Bölgeden rastgele üç eleman seçilir. Ortancası pivot yapılır.
  - Örnek: sıralı bir dizide seçilen üç elemanın ortancası 5 ise pivot 5 olur.
  - Pivot olacak eleman önce bölgenin başına alınır (5, baştaki 1 ile yer değiştirir). Sonra aynı partition işlemi uygulanır. Pivot başa alınmazsa bölme yine bozulur.

**Sıralama Algoritmalarının Karşılaştırması**

- Bin elemanda üç algoritma da anlık çalışır. Bu kadar küçük boyutta $n^2$ sorun değildir.
- Milyon elemanda insertion sort 2,8 saat sürer. Mergesort 1 saniyede, quicksort 0,6 saniyede bitirir.
- Milyar elemanda insertion sort 317 yıl sürer. Mergesort 18 dakika, quicksort 12 dakika sürer.
- $n^2$ ile $n \log n$ arasındaki fark büyük boyutlarda böyle görülür.
- Quicksort'un mergesort'a göre avantajları:
  - Ek dizi kullanmaz.
  - Biraz daha az karşılaştırma ve yer değiştirme yapar.
- Mergesort'un avantajı: en iyi, ortalama ve en kötü durumu aynıdır ($n \log n$). Quicksort'un en kötü durumu ise $n^2$'dir.
- Küçük parçalar: Özyinelemenin kendi maliyeti vardır. Bu yüzden quicksort'ta küçük bir parçaya (örneğin 50 eleman) inildiğinde o parça insertion sort ile sıralanır. Hoca Java'daki sıralamanın da böyle yaptığını söyledi. Eşik değerini ezbere bilmediğini, araştırılabileceğini belirtti.

### Mergesort için Bir İyileştirme

Son birleştirme adımında sol parça `2 4 5 7`, sağ parça `9 13 17 19` olsun.

- Normalde 2 ile 9, 4 ile 9, 5 ile 9, 7 ile 9 tek tek karşılaştırılır.
- Oysa iki parça kendi içinde sıralıdır. Sol parçanın son elemanı (7), sağ parçanın ilk elemanından (9) küçükse sol parçanın hepsi sağ parçadan küçüktür.
- Bu durumda karşılaştırma yapmadan iki parça art arda eklenebilir.

### Böl ve Yönet Üzerine Sorular

- Quicksort da böl ve yönettir. Pivot yerleştikten sonra sol parçayı sıralarken sağa bakmaya gerek yoktur. Çünkü sağda pivottan küçük eleman olamaz.
- Mergesort'ta da iki yarı birbirinden bağımsız sıralanır, sonra birleştirilir.
- İki matrisin çarpımı böl ve yönetle yapılabilir.
- Parçalar bağımsız olduğu için paralel işlemcilerde her parça ayrı bir işlemciye verilebilir. Böylece $n \log n$'e inmenin ötesinde paralellikten de yararlanılır.

### Veri Sıkıştırma ve Run-Length Encoding

- Sıkıştırmanın iki amacı vardır:
  - Veriyi daha az yerde saklamak (örneğin 1 MB'lık resmi 100 KB'ta saklamak).
  - Veriyi daha hızlı ve verimli iletmek.
- Sıkıştırma özellikle görüntü ve seste etkilidir. Taranmış bir kitap sayfasında büyük beyaz bloklar vardır.
- Birinci yaklaşım: çok kullanılan rengi az bitle, az kullanılan rengi çok bitle göstermek. JPEG ve MPEG içinde Huffman kodlaması bu fikirle kullanılır.
- İkinci yaklaşım: art arda tekrar eden değerleri "kaç tane, hangi değer" diye yazmak. Buna run-length encoding denir. "Run length", aynı değerin art arda geçme uzunluğudur.
  - Örnek: `20 20 20 20 20 20 5 5 5 5 5 5 5 2 2 2` dizisi `6 20 7 5 3 2` olarak kodlanır.
  - Çözme (decode): `6 20` görülünce yan yana 6 tane 20 yazılır.
- RLE kayıpsızdır (lossless). Kodlanıp çözülen görüntü orijinalin aynısıdır.
- Örnek görüntüler PGM (Portable GrayMap) biçimindedir. Dosyanın başında satır ve sütun sayısı yazar.
- Kodlanmış çiftleri bağlı listede tutmaya gerek yoktur. Yalnız boyutun üst sınırı baştan bellidir (en kötü durumda her piksel için bir çift), araya ekleme yapılmaz. Bağlı listenin her düğümdeki adres maliyeti boşuna olur. Düz dizi yeterlidir.

### Ders sonrası notlar

- **Tahtadaki `partition` kodunun iki sınır hatası:** Kod dersteki iki örnekte doğru çalışır. Ancak iki uç durumda dizinin dışına taşar:
  - `j` döngüsünde `a[j] >= pivot` yazılmış. Pivot bölgenin en küçük elemanıysa (örneğin dersteki en kötü durum örneği olan sıralı `1 2 3 4 5 6`) `j`, `a[l]`'yi de geçer ve `a[l-1]`'e erişir. Koşul `a[j] > pivot` olmalıdır. Bu durumda `j` en geç `l`'de durur, çünkü `a[l] = pivot`.
  - İlk turda `j = r + 1` olduğu için `i < j` koruması `i`'nin `r + 1`'e çıkmasına izin verir. Bölgedeki tüm elemanlar pivottan küçükse `a[r+1]`'e erişilir. Koruma `i < r` olmalıdır.
  - Düzeltilmiş hâl aşağıdadır. Bu kod derste yazılmadı; ajan tarafından eklendi. `gcc -Wall -fsanitize=address,undefined` ile derlendi. Dersteki iki örnekte tahtadaki sonuçları verdi (`1 2 3 7 4 8 5` ve `3 2 5 6 7 8`). Sıralı diziyi ve 20.000 rastgele diziyi (tekrarlı değerlerle) doğru sıraladı. Tahtadaki hâl ise sıralı dizide bellek hatası verdi.

```c
void xchg(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int l, int r)
{
    int pivot = a[l], i = l, j = r + 1;
    do {
        do { i++; } while ((i < r) && (a[i] < pivot));
        do { --j; } while (a[j] > pivot);
```

```
        xchg(&a[i], &a[j]);
    } while (i < j);
    xchg(&a[i], &a[j]);   /* son yer değiştirmeyi geri al */
    xchg(&a[l], &a[j]);   /* pivotu yerine koy */
    return j;
}

void quickSort(int a[], int l, int r)
{
    if (l < r) {
        int s = partition(a, l, r);
        quickSort(a, l, s - 1);
        quickSort(a, s + 1, r);
    }
}
```

- **Prim ve MST'nin tekliği:** Hoca, Prim algoritmasında başlangıç düğümüne göre farklı MST'ler elde edilebileceğini söyledi. Kenar ağırlıkları birbirinden farklıysa bu doğru değildir. MST tektir; Kruskal da, Prim de (hangi düğümden başlanırsa başlansın) aynı ağacı bulur. Quizdeki 3. ifadenin doğru olmasının gerçek sebebi ağırlıkların farklı (distinct) olmasıdır. Eşit ağırlıklı kenarlar varsa birden fazla MST olabilir; bu durumda iki algoritma da eşitlikleri nasıl çözdüğüne göre farklı ağaçlar verebilir.
- **Tahtadaki `partition` kodunda ayrıca:** tanımlı `t` değişkeni hiç kullanılmıyordu (`gcc -Wall` uyarısı). Yukarıda çıkarıldı.
- **Faktöriyel ve böl ve yönet:** Hocanın "önceki sonuca ihtiyaç var, böl ve yönet olamaz" gerekçesi faktöriyel için doğru değildir. $n!$, $1 \cdot \ldots \cdot n/2$ ve $(n/2+1) \cdot \ldots \cdot n$ diye iki bağımsız parçaya bölünebilir. Ama toplam çarpma sayısı yine $n-1$'dir, kazanç olmaz. Bu yüzden "uygun değil" sonucu ikinci şarttan gelir: bölmek kazanç sağlamaz.
- **İkinci en küçük kenar:** Hoca MST'nin "ikinci en küçük ağırlığı belki içermeyebilir" der (00:07:14). Ağırlıklar farklı ve paralel kenar yoksa ikinci en küçük kenar da her zaman MST'dedir. İlk iki kenar döngü oluşturamaz, Kruskal ikisini de alır.
- **Tüm kenarlara 10 eklemek:** Her yayılan ağaçta tam $V-1$ kenar vardır. Bu yüzden her ağacın toplamı aynı miktarda ($10(V-1)$) artar ve sıralama değişmez.
- **"Quicksort'ta en iyi, en kötü ve ortalama durum eşittir":** Hoca bunu önce quicksort için söyledi, hemen ardından mergesort'u kastettiğini belirtti. Doğrusu: mergesort'ta üçü de $n \log n$'dir. Quicksort'ta en kötü durum $n^2$'dir.
- **Ortalama durum katsayısı:** Quicksort'un ortalama karşılaştırma sayısı yaklaşık $2n \ln n \approx 1{,}39\, n \log_2 n$'dir.
- **Quicksort'un ek yeri:** Quicksort ek dizi kullanmaz. Ancak özyineleme yığını (stack) ortalamada $O(\log n)$, en kötü durumda $O(n)$ yer tutar.
- **Matrisin $n$'inci kuvveti:** Hoca bunun böl ve yönete girmediğini söyledi. Ancak $A^n$, $A^{n/2}$ bir kez hesaplanıp karesi alınarak $O(\log n)$ matris çarpımıyla bulunabilir (tekrarlı kare alma, repeated squaring). Bu yöntem kaynaklarda azalt ve yönet (decrease and conquer) ya da böl ve yönet örneği olarak geçer. Aynı yöntemle tek bir Fibonacci sayısı da $O(\log n)$'de hesaplanabilir. Quizdeki soru ise $n$'e kadar **tüm** Fibonacci sayılarını üretmekti; bu iş sıralıdır ve böl ve yönete uygun değildir.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.